

38. Principy a struktury správy souborů a správy paměti

38. Správa souborů a správy paměti

SZZ okruh 38 (FIT BUT). Jak OS ukládá soubory na disk a mapuje paměť.

Shrnutí

Souborové systémy

- **Soubor** = jednotka uchování dat; **sektor** (nejmenší adresovatelný), **alokační blok** (2^n sektorů); fragmentace (interní/externí).
- **Unix i-uzel** — metadata + odkazy na data (10 přímých + nepřímé 1./2./3. úrovně); **i-uzel neobsahuje jméno souboru** (to je v adresáři).
- Organizace dat: kontinuální, zřetězené seznamy, **FAT** (tabulka odkazů), **B+ stromy** (sekvenční i náhodný přístup), **extenty**, NTFS (MFT).
- **Žurnálování** (REDO/UNDO), copy-on-write, RAID (0 striping, 1 mirror, 5/6 parita).

Správa paměti (stránkování)

- **LAP** (logický, stránky) $\times$ **FAP** (fyzický, rámce); **MMU** překládá LAP $\rightarrow$ FAP (zrychlení **TLB**).
- Přidělování: spojitě bloky, segmenty, **stránkování** (stránka $\rightarrow$ rámec, eliminuje externí fragmentaci, způsobuje interní).
- Tabulky stránek: jednoúrovňové (nepraktické), **hierarchické** (4úrovňové), hashované, invertované.

Výpadky a výběr oběti

- **TLB hit/miss** ($\neq$ výpadek stránky); **výpadek stránky** (page fault) $\rightarrow$ MMU přerušení $\rightarrow$ jádro načte/alokuje rámec.
- Stránkování **na žádost** (demand paging), swap; **trashing** při nedostatku rámců.
- Algoritmy výběru oběti: **FIFO** (Beladyho anomálie), **LRU** (nejdéle nepoužitá, HW podpora).

Latence a princip lokality viz [[okruhy/04-hierarchie-pameti]]; B+ stromy viz [[okruhy/30-vyhledavani-razeni]]; procesy/IPC viz [[okruhy/39-planovani-synchronizace-procesu]].

Související syntéza

- [[synthesis/vyhledavaci-b-stromy|Vyhledávací stromy $\times$ paměťová hierarchie]] — syntéza
- [[synthesis/transakce-acid-db-os|Transakce a ACID: DB $\times$ OS]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Souborové systémy □ [[#Souborové systémy]] - *Co je i-uzel, obsahuje jméno?* □ Datová struktura s metadaty a odkazy na data; **jméno NE** (je v adresáři). - *FAT vs. B+ stromy?* □ FAT = tabulka odkazů (průchod sekvenční); B+ stromy = log. složitost, sekvenční i náhodný přístup.

Správa paměti □ [[#Správa paměti (stránkování)]] - *LAP vs. FAP, MMU?* □ Logický (stránky, per proces) × fyzický (rámce, jeden); MMU mapuje LAP → FAP. - *Co je stránkování?* □ LAP po stránkách □ rámce stejné velikosti (4 KiB); eliminuje externí fragmentaci. - *TLB?* □ Asociativní cache mapování stránka → rámec; TLB hit = 1 přístup, miss = hledání v tabulce stránek.

Výpadky stránek □ [[#Výpadky a výběr obětí]] - *Co je výpadek stránky?* □ Proces odkazuje stránku bez přiděleného rámce □ MMU přerušení □ jádro alokuje/načte ze swapu. - *FIFO vs. LRU?* □ FIFO odloží nejstarší (Beladyho anomálie); LRU nejdéle nepoužitou (lepší, vyžaduje HW).

Plné znění (ke studiu)

Organizace a ukládání souborů

- **Soubor** - Základní organizační jednotka pro **uchovávání dat** na vnějších paměťových médiích.
- **Souborový systém** - Souhrn pravidel definujících chování a vlastnosti jednotlivých souborů a možnosti jejich další logické organizace. Také definuje způsob uložení požadovaných dat a informací k nim.
 - **FAT** - Univerzální mezi OS, je totiž skoro všude podporovaný.
 - **EXT** (2,3,4) - Pro linux.
 - **NTFS** - Pro Windows.

Uložení dat na HDD

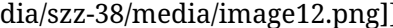
[[media/szz-38/media/image16.png]]

Disk je rozdělen na sektory, což jsou nejmenší jednotky, které lze číst/zapsat (dříve 512B dnes 4096B). Adresace sektorů:

- **Cylindr-Hlava-Sektor (CHS)** - Adresace systémem souřadnic, které jsou specifikované číslem válce (C, válec je tvořen z track na více platters), hlavou (H) a sektorem (S), který se nachází na dráze (track). Vhodné pro menší disky, jelikož větší vyžadují **proměnný počet sektorů** na stopě v závislosti na čísle válce (vzdálenosti od středu) a adresace se děje v OS. [[media/szz-38/media/image17.png]]
- **Logical Block Addressing (LBA)** - Adresace pomocí lineárního logického čísla sektoru (od 0 do N). Používá se v dnešní době, samotné vyhledání bloku na disku řeší nějaký obvod v disku, nikoliv OS.

Pojmy

- **Sektor disku** - Jeho nejmenší adresovatelná jednotka (má pevnou délku).
- **Alokační blok - 2n sektorů** - Nejmenší jednotka diskového prostoru, se kterou dovoluje OS pracovat.
- **Fragmentace** - Data jsou uložena nesouvisle po částech. Zpomaluje operace prováděné s diskem. Disk lze defragmentovat - přeuspořádat uložená data, aby byla souvislá.
 - **Interní** - Fragmentace uvnitř alokovaných oblastí. Souborový systém **vyhradí pro soubor větší prostor**, než je jeho velikost.

- **Externí** - Fragmentace mezi alokovanými oblastmi, vzniká mazáním souborů - vytvořením nesouvislého uložení. Při nedostatku místa musí být soubor rozdělen (fragmentován) na části, které jsou uloženy do volných míst. Stejný problém vzniká při zvětšování souborů (za zvětšeným souborem může být uložen jiný soubor a je nutné jej rozdělit). Může to zapříčinit, že na disk nelze uložit soubor, i když je tam dostatek místa (soubor nelze ukládat po částech nebo volná místa jsou natolik malá, že není možné současně uložit metadata a data, tj. je nevhodně zvolená velikost alokačního bloku).
- **Přístup na disk (čtení a zápis)** - Přístup se musí plánovat v závislosti na aktuální pozici hlav. Jednotlivé požadavky lze přeuspořádat tak, aby se hlavy musely mezi přístupy pohybovat co nejméně. Různé postupy při pohybu hlav (od středu k okraji a zpět, pouze v mezikruží, kde jsou požadovaná data, operace prováděné pouze při pohybu v jednom směru, ...). Dokončení operace (včetně chyb) je oznamováno pomocí HW přerušování. 
- **Logický disk** - Dělení fyzického disku na diskové oddíly (**partition**), se kterými je možné nezávisle manipulovat (jeví se jako více fyzických disků). Tabulka MBR (**Master Boot Record**) nebo novější GPT (**GUID Partition Table**) obsahuje informace o diskových oblastech.
- **Žurnálování** - stejně jako u DB zajišťuje zachování konzistence dat na disku při zápisu. Na žurnál se zapisují operace, které budou prováděny, a po úspěšném zapsání na disk se odstraní. Při chybě lze použít jednu ze dvou metod REDO (operace byla dokončena, ale všechna data ještě nebyla zapsána na disk) a UNDO (operace začala upravovat data, ale nebyla dokončena).
- **Copy-on-write** - nejprve zapisuje nová data a metadata na disk, poté je až zpřístupní. Zápis dat vychází z uložení v B+ stromech. Data se zapisují postupně od listového uzlu, kde jsou uložena, až po kořen (vnitřní uzly představují metadata). Tedy nejprve máme původní data, potom někde na disk zapíšeme nová data (původní data stále necháváme včetně vnitřních uzlů), postupně od listů ke kořeni upravujeme vnitřní uzly. Až když máme všechny vnitřní uzly upravené, nahradíme původní uzly novými uzly, které už ukazují na nová data.

Typické parametry disku

- kapacita do 20 TB,
- doba přístupu od nízkých jednotek ms,
- přenosová rychlost v desítkách až stovkách MB/s

RAID (Redundant Array of Independent Disks):

Metoda pro zabezpečení dat proti selhání pevného disku.

- **RAID 0:** Disk striping - následné bloky dat jsou rozmístěny na více discích, což znamená vyšší výkonnost, ale žádnou redundanci □ není to vlastně raid, nechrání před poruchou.
- **RAID 1:** Disk mirroring - data se ukládají na **2 disky**, velká redundance a výkonnost stejná jako u 1 disku.
- **RAID 2** - Data jsou na discích rozdělena po bitech a zabezpečena **Hammingovým kódem**. **Menší redundance** dat než u RAID1.
- **RAID 3-5** - Používají paritu, dokáží se vyrovnat se **ztrátou 1 disku**.
- **RAID 6** - Používá 2 paritní bloky oproti RAID 5. dokáže se vyrovnat se **ztrátou 2 disků**.

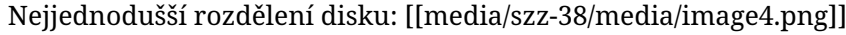
Parita se používá se k jednoduché detekci chyb

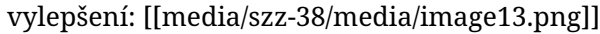
- **Lichá** - Lichý počet jedniček.
- **Sudá** - Sudý počet jedniček.

Uložení dat na SSD

SSD jsou organizovány do **stránek** (4096 B - 4 KiB) a ty do **bloků** typicky po 128 stránkách (tj. 512 KiB). Prázdné stránky lze přepisovat jednotlivě, pro přepis stránky (editace, mazání) je nutné načíst **celý blok** (512 KiB) do VP, provést požadované změny, z disku tento blok dat **smazat a zapsat jej znovu**. Problém je řešen řadičem SSD, který může sám stránky přesouvat a uvolňovat celé bloky. SSD také může obsahovat nějaké stránky nad udávanou kapacitu, které jsou použity pro přepis.

Unixový systém souborů FS

Nejjednodušší rozdělení disku: 

vylepšení: 

- disk je rozdělen do skupiny **bloků**, každá skupina má **své i-uzly** a datové bloky a volné bloky. Zajišťuje lepší lokalitu dat.
- super blok s informacemi o FS je ukládán vícekrát (při poškození tohoto bloku je problematické číst z disku obnovit data, proto redundance)

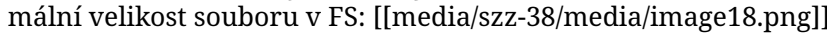
i-uzel (inode)

Základní **datová struktura popisující soubor** v UNIXových souborových systémech. Obsahuje metadata, ve speciálních případech i data (např. symbolický odkaz - pokud je cesta krátká je uložena přímo v i-uzlu). Metadata tvoří:

- **stav i-uzlu** (alokovaný, volný, ...),
- **typ souboru** (obyčejný, adresář, **zařízení**, např. **dev/tty** (linux používá 2 abstrakce, a to soubory a procesy), symbolický odkaz, ...)
- **délka souboru v bajtech**,
- **mtime** = čas poslední modifikace dat,
- **atime** = čas posledního přístupu,
- **ctime** = čas poslední modifikace i-uzlu,
- **UID** = identifikace vlastníka (číslo),
- **GID** = identifikace skupiny (číslo),
- **přístupová práva** (číslo, například **0644** znamená **rw-r--**).

Samotná data jsou odkazována pomocí:

- **10 přímých** odkazů na data (rychlé, ale jen pro malé soubory),
- **1 nepřímý odkaz první úrovně**,
- **1 nepřímý odkaz druhé úrovně** (tyto bloky obsahují pouze odkazy na bloky nepřímých odkazů první úrovně),
- **1 nepřímý odkaz třetí úrovně** (tyto bloky obsahují pouze odkazy na bloky nepřímých odkazů druhé úrovně),

Na základě **velikosti jednotlivých bloků** a **velikosti odkazů na bloky** lze poté vypočítat maximální velikost souboru v FS: 

Velikost souboru je také ovlivněna OS (např. na 32 bitech je největší reprezentovatelné číslo $2^{32}-1$, což odpovídá 4 GiB). **i-uzel NEOBSAHUJE název souboru.**

Jiné způsoby organizace souborů

Organizace souborů a popis uložení jsou implementovány tak, aby byla **minimalizována režie** při práci s nimi a **byly snadné** následující úkony:

- **průchod souboru**, což je spojené s rychlostí **vyhledání následujícího bloku**,
- **přesun v souboru** (seek), což je spojené s **rychlostí vyhledání prvního a určitého bloku** souboru,
- **zvětšování a zmenšování souboru**, což je spojené s **přidáním bloků** (alokací prostoru) a **odebráním bloků** (dealokací prostoru).

Kontinuální uložení

Jednoduchá data jsou uložena na disku jako jedna **spojitá posloupnost**. Problematické je **zvětšování souborů**, pokud se za souborem nachází další. Problém ukládání nových souborů při **externí fragmentaci**.

Zřetěžené seznamy bloků

Každý datový blok kromě dat obsahuje odkaz na další blok (nebo příznak konce souboru). Problém je náhodný přístup k určité části souboru. Vždy se musí **procházet sekvenčně** od začátku. Stejný problém je při zápisu. Chyba v provázání kdekoliv na disku vede ke ztrátě zbytku souboru. Řeší ale problém s fragmentací (blok lze ukládat na disk libovolně nespojitě).

File Allocation Table (FAT)

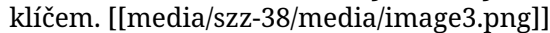
[[media/szz-38/media/image10.png]]

Podobný princip zřetěženým seznamům bloků. Tentokrát ale **není odkaz** na následující blok uložen **na konci datového bloku**, ale ve speciální **tabulce** (FAT - ta je uložena na začátku disku a pro jistotu 2×). Buňka tabulky obsahuje **odkaz** (index) **na jinou buňku** v tabulce a **odkaz** (index) **na blok dat** na disku. **Částečně** řeší problém s náhodným přístupem, není nutné načítat do paměti blok po bloku (pro nalezení odkazu na další), ale samotný průchod ve FAT je **pořád sekvenční** (bude ale rychlejší).
[[media/szz-38/media/image5.png]]

B+ stromy

Dnes nejběžnější způsob ukládání souborů na disk (s případnými modifikacemi). **Řeší problém sekvenčního i náhodného přístupu** k datům souboru. Efektivnost B+ stromů je založena na **logaritmické složitosti** vyhledávání ve stromě. Struktura stromu:

- **vnitřní uzel**: je tvořen **k** klíči (identifikátory bloků - jejich sekvenční číslo) a **k+1** odkazy na bloky nižší úrovně (tedy i listové). Klíče jsou v uzlu uspořádané **od nejmenšího k největšímu** (obecně mezi nimi musí existovat nějaká relace uspořádání). Po sobě následující klíče vymezují intervaly klíčů, které jsou získatelné, pokud bude následován odkaz mezi těmito klíči. (na začátku uzlu si lze představit imaginární klíč jehož hodnota je rovna min a na konci klíč s hodnotou max+1). Tyto **intervaly jsou zleva uzavřené a zprava otevřené**, tzn. že podmínka jestli je klíč v daném intervalu, vypadá následovně: **if (key_i <= key_hledany < key_i+1)** tak následuj daný odkaz (napravo od **key_i**), jinak posuň index porovnávaných klíčů. Protože jsou klíče seřazené, šlo by použít i vyhledávání založené na půlení intervalů.
- **listový uzel**: mají stejnou strukturu, odkazy ale vedou přímo na datové bloky (data souboru). **Poslední odkaz listového uzlu odkazuje na následující listový uzel**, což řeší sekvenční průchod souboru. Posloupnost hodnot klíčů na listové úrovni ale **nemusí být spojitá** (fragmentace), musí se zde nacházet **všechny klíče** (sekvenční čísla bloků souborů). Výběr odkazu na

datový blok pak proběhne tak, že **porovnáváme** hodnotu hledaného klíče a hodnoty klíčů uložených v listovém uzlu na **rovnost**, pokud jsou si rovny, použijeme **odkaz na levo** od klíče, na kterém došlo ke shodě: **if (key_i == key_hledany) pak použij link_i**, jinak porovnávej s dalším klíčem. 

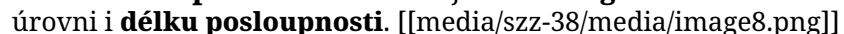
B+ strom je **výškově vyvážený** a dále pro něj platí:

- **Limity zaplnění:** pro uzly s **m** odkazy (tedy klíči **key_1** až **key_m**)
 - sólo kořen **1** až **m**,
 - kořen **2** až **m**,
 - vnitřní uzel $\lceil m/2 \rceil$ až **m**,
 - list $\lceil m/2 \rceil$ až **m**.
- **Vkládání:** vkládá se na **listové úrovni** při **přepnutí** se uzel **rozštěpí** a případně se rozštěpí i uzly ve vyšších úrovních až nakonec se může rozštěpit kořen, což vede na přidání nového kořene a **zvětšení výšky stromu** o jedna.
- **Rušení:** ruší se od listové úrovně a pokud je **porušena minimální naplněnost**, dojde buď k **přerozdělení** odkazů **mezi sourozenci** (pokud je to možné), nebo ke **sloučení** sousedních uzlů, a případně ke sloučení uzlů ve vyšších úrovních, až nakonec může dojít ke sloučení druhé úrovně a zrušení původního kořene. Výška stromu se tak **sníží** o jedna.

Extenty

Extenty odkazují na proměnný počet bloků, které jsou **za sebou** uloženy **logicky** (tj. v souboru) a také **fyzicky** (tj. na disku). Zrychluje práci s velkými soubory a snižuje množství metadat. Extenty se **kombinují s B+** stromy, **nelze** je ale použít v **Unixovém stromu**, protože zde není mechanismus, jak ukládat informaci o jejich proměnné délce.

Strom extentů

Analogie B+ stromu **bez vyvažování** a **bez zřetězení listů**. Má omezený počet úrovní **max. 5 úrovní**. Kořen je umístěn **do i-uzlu** a má **max 4 odkazy** (stačí pro malé soubory nebo při málo zaplněném disku), vnitřní uzly mohou mít **i více**. Vnitřní **indexové** uzly se vytváří až při naplnění odkazů v kořeni, poté se ale do nově vytvořeného indexového uzlu přesunou všechny a do kořene se umístí odkaz na tento indexový uzel. Indexové uzly obsahují hlavičku, ve které je **aktuální počet odkazů** a **maximální počet odkazů** a dvojice **číslo logického bloku** a **číslo fyzického bloku** a na listové úrovni i **délku posloupnosti**. 

NTFS (New Technology File System)

Souborový systém vyvinut pro Windows NT. Využívá **Master File Table** (MFT), ve které je alespoň jeden řádek pro každý soubor. Obsah souboru poté může být:

- uložen **přímo** v MFT daného souboru,
- odkazován z MFT pomocí **extentů**,
- odkazován z MFT pomocí **B+ stromu**, který je tvořen jinými MFT záznamy odkazovanými z **primárního MFT** záznamu.

Organizace volného prostoru

- **bitové pole** (bitová mapa) s jedním **bitem pro každý blok**, vyhledávání pomocí bitového maskování.

- **zřetězení** volných bloků,
- **zřetězení odkazů** ve FAT, které odkazují na volné bloky,
- **B+ stromem** (adresace velikostí nebo offsetem),
- Po **extentech**.

Deduplikace

Zamezení opětovnému ukládání stejných dat (na úrovni souboru, extentů, bloků, bytů). Může (výrazně) **ušetřit místo** např. na mail serverech (stejná příloha u několika emailů), git repozitářích (malé změny v souborech). Lze implementovat při **zápisu** nebo **dodatečně**, využívá se **kryptografického hashování** pro hledání shody (a poté případného porovnání). Problém nastává při změně jednoho ze stejných souborů, nutno rozdvojit atd. Při **malé duplikaci** může způsobovat navýšení spotřeby CPU času i místa na disku.

Typy souborů

[[media/szz-38/media/image22.png]]

- **Adresáře** obsahuje dvojice **jméno souboru** (dříve 14 znaků, dnes až 255, nemůže obsahovat / a \0) a **číslo souboru** (typicky se jedná o **číslo i-uzlu**, případně číslo pro hledání v B+ stromu). Adresář vždy obsahuje odkazy na sebe (.) a rodiče (..). Implementace pomocí seznamů, B+ stromů, hash table.
- **Symbolický odkaz** obsahuje jako data jméno odkazovaného souboru, při otevření se vícekrát musí zpracovávat cesta. Po smazání odkazovaného souboru **symlink** zůstává ale je neplatný (jeho otevření způsobí chybu). Cykly se řeší omezením úrovně zanoření. Cesta a jméno odkazovaného uzlu může být uložena přímo v i-uzlu.
- **Terminály** jsou fyzická nebo virtuální rozhraní umožňující primárně textový výstup, editaci na vstupním řádku a speciální znaky. V UNIX jsou reprezentovány soubory (/dev/tty, /dev/tty1, ...). Více režimů zpracování znaků (**raw**, **cbreak**, **cooked**).
- **Roury** (pipes) umožňují **komunikaci mezi procesy**, mají **dva deskriptory** (čtecí a zápisový). V terminálu se vytváří pomocí znaku |. Jsou implementovány pomocí kruhového bufferu s omezenou kapacitou. Dělíme je na **pojmenované roury** a **nepojmenované roury**.
- **Sockety** Umožňují síťovou i lokální komunikaci mezi procesy (pojmenované a uložené v FS). Komunikace může být **blokující** i **neblokující**.

Mountování disků

Nový disk (a jiné zařízení) lze namountovat do libovolného adresáře již existujícího adresářového stromu. Dnes většina OS automaticky mountuje disky při připojení do PC.

Virtual File System (VFS)

Vytváří **jednotné rozhraní** pro práci s **různými souborovými systémy**, odděluje vyšší vrstvy OS od konkrétní implementace jednotlivých souborových operací na jednotlivých souborových systémech (FAT, ext4, NFS, ...). [[media/szz-38/media/image19.png]]

Network File System (NFS)

Zpřístupňuje soubory uložené na vzdálených systémech, zapojuje se do **VFS** a práce s ním je pak pro OS i uživatele stejná jako se soubory na disku.

Spooling (simultaneous peripheral operations on-line)

[[media/szz-38/media/image15.png]]

Umožňuje **zrychlení** u pomalých **výstupních zařízení**. Výstup je proveden do **souboru** a proces, který ho zadal (nejčastěji se jedná o **tisk**), může pokračovat. Vytvořený soubor je uložen do **fronty požadavků** na výstupní zařízení (tiskárnu) a čeká, až na něj přijde řada.

Přístupová práva

Různá přístupová práva pro **vlastníka**, **skupinu** a **ostatní**.

např. **-rwx—r—** (číselně 0704), první znak znamená typ souboru (zde obyčejný soubor)

- **vlastník**: čtení, zápis, provedení, [[media/szz-38/media/image9.png]]
- **skupina**: nemá žádná práva,
- **ostatní**: pouze čtení.

Sticky bit (t) je příznak, který nedovoluje rušit a přejmenovávat cizí soubory v adresáři, i když mají všichni právo zápisu (-rwxrwxrwx~~t~~).

Práva pro procesy

- **UID** - ID uživatele, který ho spustil.
- **EUID** - Efektivní UID - pro kontrolu přístupových práv.
- **GID** - ID skupiny, ve které je uživatel, který proces spustil.
- **EGID** - Efektivní GID - Podobně jako EUID.
- **SUID/SGUID** - Pro propůjčení práv vlastníka uživateli.

Práce se soubory vstup/výstup/mazání

Pro urychlení práce se soubory se používají vyrovnávací paměti (VP), které minimalizují operace s pomalými periferiemi (HDD, SSD, terminál, ...). Dílčí VP mívají **velikost datového bloku** nebo skupiny a jsou sdružovány do kolekcí pevné nebo proměnné délky. Možná implementace pomocí **hash table**.

Čtení

[[media/szz-38/media/image25.png]]

Postup při prvním čtení (soubor ještě není ve VP):

1. Přidělení VP a načtení bloku (prvního nebo požadovaného)
2. Kopie požadovaných dat z VP do adresového prostoru procesu (RAM→RAM).

Dokud jsou data v RAM a dochází pouze k bodu **2**, jakmile čtením proces **překročí do jiného datového bloku** dochází k bodu **1 i 2** (pokud ještě není ve VP).

read()

1. Kontrola platnosti **fd**.
2. Provedení kroků 1 a 2 popsaných výše dle potřeby.
3. Návrátovou hodnotou je **počet doopravdy přečtených** bytů nebo -1 při chybě a nastavení **errno**.

Zápis

Postup:

1. Přidělení VP a načtení bloku souboru do VP (pokud se nevytváří nový nebo zcela nepřepisuje).
2. Zápis dat do VP z adresového prostoru procesu (RAM → RAM), **nastavení příznaku** modifikace (**dirty bit**).
3. **Zpožděný** zápis na disk, **nuluje příznak** (např. při uvolnění místa z RAM, ...).

write()

1. Kontrola platnosti **fd**.
2. Provedení kroků 1 a 2 popsaných výše dle potřeby.
3. Před zápisem dojde ke **kontrole dostupnosti dostatečného prostoru** na disk.
4. Návrátovou hodnotou je **počet doopravdy zapsaných** bytů nebo -1 při chybě a nastavení **errno**.

Otevření

Nejdříve (v průběhu bodu 1) se provádí kontrola na přístupová práva.

Soubor ještě **nebyl otevřen**:

1. **hledání i-uzlu** souboru: na základě cesty a jména souboru se postupně načítají **i-uzly adresářů na cestě** a datové bloky s obsahem adresářů, až je nakonec vyhledán požadovaný soubor a jeho i-uzel. Některé (časté) i-uzly mohou být už ve VP.
2. V **systémové tabulce aktivních i-uzlů (inode table)** se vyhradí nová položka, do které se načte z VP **i-uzel** a stává se z něj tak **v-uzel**, což je jeho **rozšířená kopie**.
3. V **systémové tabulce otevřených souborů (open file table)** vyhradí novou položku a naplní ji:
 1. **odkazem** na položku tabulky **v-uzlů**,
 2. **režimem** otevření (čtení, zápis - smaže původní obsah, čtení i zápis),
 3. **pozicí** v souboru (0),
 4. **čítačem počtu referencí** na tuto položku (1), počet referencí roste pouze při **fork** nebo **dup**, kdy dojde k **duplikaci fd** pro nový proces, procesy si mohou měnit pozici v souboru - nebezpečné...
4. V **poli deskriptorů souborů** (každý proces má vlastní, obsahují informace o **fd**) se vyhradí položka (**fd - int, index do pole**) a naplní se odkazem na položku v tabulce otevřených souborů.
5. Vracení odkazu na **fd** nebo chyba. [[media/szz-38/media/image1.png]]

Otevření pro čtení **již otevřeného** souboru (např. jiný proces nebo i ten samý):

1. **Vyhledání čísla i-uzlu**, jako při novém otevření.
2. **Vyhledání v-uzlu** v systémové tabulce (provádí se i při prvním otevření, ale při tom vyhledání selže, tabulka musí být uzpůsobena vyhledávání).
3. **Vyhrazení nového záznamu v systémové tabulce otevřených souborů** (může jít o jiný způsob otevření a také každé otevření může vyžadovat jinou pozici v souboru, odkaz v této tabulce se sdílí **pouze po fork**) s odkazem na v-uzel a **zvýšení počítadla odkazů v-uzlu**.
4. Vytvoření nového **fd**.
5. Vrácení **fd** nebo chyba.

Přímý přístup

[[media/szz-38/media/image7.png]]

Pomocí lseek, postup:

1. Kontrola platnosti **fd**,
2. nastavení **pozice fd jako offset bytů**,
3. Návratovou hodnotou je **výsledná pozice** nebo -1 při chybě

Při seek za konec souboru a zapsáním vzniká řádký soubor (uprostřed má prázdný prostor, který není uložen na disku), offset může být záporný, nelze ale nastavit pozici před začátek souboru.

Zavření souboru

Zavření souboru **nezpůsobí uložení obsahu jeho VP na disk**. Postup:

1. Kontrola platnosti **fd**.
2. Uvolnění **fd** z tabulky fd, snížení odkazů na záznam v **tabulce otevřených souborů** (open file table).
3. Pokud klesne počet referencí na záznam v tabulce otevřených souborů na 0, je záznam odebrán a snížen počet referencí na v-uzel v **tabulce otevřených i-uzlů**.
4. Pokud klesne počet referencí na v-uzel na 0, z v-uzlu se **okopíruje část, která tvoří i-uzel do VP** a v-uzel se uvolní.
5. 0 jako úspěch -1 neúspěch.

Mazání/rušení souboru

1. **Vyhodnocení cesty** a existence souboru, **kontrola přístupových práv**.
2. **Odstranění pevného odkazu** na i-uzel, v daném adresáři (vyžaduje práva pro zápis do adresáře).
3. **Zmenšení počtu odkazů** (jmen) na i-uzel (symlink v tomto počtu nejsou zahrnuty)
4. Pokud počet jmen **klesne na 0**, může být i-uzel a všechna jeho **data uvolněna**, to se provede, až soubor přestanou všichni používat (bude všemi uzavřen). Tzn. soubor jde smazat vždy (ne jak na Windows, kdy je kvůli tomu potřeba pomalu restartovat PC...), to platí i pro spustitelné soubory (program dále poběží).

Správa paměti

Rozlišujeme:

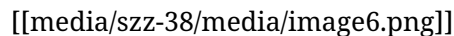
- **Logický adresový prostor (LAP):** jedná se o virtuální adresový prostor, se kterým pracuje procesor při provádění kódu, každý proces i jádro jej **má svůj** a jsou **stejné** (více stejných logických adres je mapováno na různé fyzické adresy v paměti.). Pro úsek paměti se používá označení **stránka** (page).
- **Fyzický adresový prostor (FAP):** pouze **jeden pro celý PC** (společný pro všechny procesy i jádro), jedná se o adresový prostor **fyzických adres v paměti** (RAM). Pro úsek paměti se používá označení **rámec** (frame). 

Memory management unit (MMU)



HW jednotka, která zajišťuje překlad logických adres na fyzické adresy, běžně je součástí čipu (na obr. ilustrativně mimo). Pro překlad MMU využívá speciálních registrů a i hlavní paměti. Pro urychlení překladu používá různé VP, např. **Translation Lookaside Buffer (TLB)**.

Přidělování paměti



Přidělování **FAP** pro zmapování do **LAP** se používají úseky paměti o velikosti, které jsou **konzistentní** se způsobem **překladu LAP** na **FAP**, který je podporován v HW daného systému, což jsou:

- **spojité bloky**,
- **segmenty**,
- **stránky**,
- kombinací přístupů.

Při programování (např. v C malloc) se přidělují **již namapované** úseky **LAP**.

Contiguous Memory Allocation

Procesům jsou přidělovány **spojité bloky** paměti o určité velikosti. Pokud není aktuálně volný **dostatečně velký** spojitý úsek paměti, proces buď musí čekat, nebo musí dojít k **odložení** celé paměti jiného procesu na HDD.

- **výhody:** velmi jednoduchá implementace v HW i SW.
- **nevýhody:** výrazný projev **externí fragmentace**, a to z důvodu přidělování a uvolňování paměti různé velikosti. Vznikají úseky, které mohou být příliš malé, na to, aby byly využity. Dalším problémem je **zvětšování přiděleného prostoru** pokud již za úsek aktuálně přidělené paměti není místo, je nutné celý již přidělený úsek přesunout do dostatečně velké oblasti a poté až přidělenou paměť zvětšit. Lze použít nějakou **strategii** pro přidělování paměti (first fit, best fit, worst fit), to ale ubírá na jednoduchosti implementace.

Přidělování paměti po segmentech

LAP je rozdělen na **několik segmentů**. Segmenty přiděluje různým částem procesu (zásobník, hromada, nebo i jednotlivým procedurám, ...) překladač nebo programátor. U segmentů může být dále specifikován způsob přístupu - čtení/zápis (bezpečnější, kód lze umístit do read only segmentu). **Logická adresa** je tvořena z **čísla segmentu** a **posunu v něm**, každý segment má číslo a velikost.

- **výhody:** poměrně jednoduchá implementace, zlepšuje externí fragmentaci, ale problém vlivem proměnné velikosti segmentů přetrvává.
- **nevýhody:** **segmentování** programu je řízeno při tvorbě programu, což může vést na **problémy při implementaci**. Stále neřeší **problém s externí fragmentací**, příliš velké segmenty nemusí být možné namapovat do **FAP**, nutné odkládat větší úseky paměti na HDD.

Stránkování (přidělování paměti po rámcích)

LAP je rozdělen na **stránky** (pages) o **pevné velikosti** (každá stránka je stejně velká, obvykle 4096 B, což odpovídá velikosti bloku na disku), **FAP** je rozdělen na **rámce**, které jsou ale **stejně velké jako stránky**. Mapuje se vždy **jedna stránka na jeden rámec** (nelze aby se jedna stránka mapovala přes dva rámce) a **mapování je odstíněno** od implementace programu a běžícího procesu. Pro zvýšení efektivity přístupu do paměti je snaha o **mapování spojitých posloupností rámců**, pokud je to možné.

- **výhody:** eliminuje externí fragmentaci (každý rámec lze využít), neviditelný pro běžící proces. Umožňuje **jemné odkládání** po stránkách/rámcích, proces tím ani nemusí být zpomalen, pokud danou stránku nepoužívá. Další výhodou je řízení přístupu pro každou stránku.
- **nevýhody:** složitější implementace (HW i SW), představuje větší režii (hledání, jestli je stránka namapovaná) a může být pomalejší (zejména při TLB miss). Stránkování způsobuje **interní fragmentaci** (přidělení větší části paměti, než je potřeba, zarovnávání, aby nebyly instrukce/data na rozmezí dvou stránek atd.)

Jednourovňové tabulky stránek Nejjednodušší, ale prakticky **nepoužitelný** způsob implementace tabulek stránek. OS si udržuje informace o volných rámcích a pro každý proces včetně jádra si udržuje jeho **tabulku stránek** (page table), **adresa tabulky** je uložena v k tomu určenému **registru**. Tabulka stránek obsahuje **popis mapování** (dvojice první logická adresa stránky a první fyzická adresa rámce) a **různé příznaky** (modifikace, přístupová práva, příznak globality - může používat více procesů, ...). Tabulky stránek jsou stejně jako data **udržovány v hlavní paměti**, z čehož plyne důvod nepraktičnosti jednourovňových stránek. Na **32 bit** systému existuje 2^{32} adres, pokud má stránka velikost **4096 B** (2^{12} B) je nutné uchovávat informaci o 2^{20} stránkách, jeden záznam o mapování stránky na rámec může mít **4 B**, což znamená **4 MiB** pro uložení stránek **pro jeden proces**. Problém exponenciálně roste u 64 bit systémů.

Při provádění příkazu v programu dochází ke **dvěma přístupům do paměti - instrukce** a její **operandy**. To vede ke dvěma přístupům do tabulky stránek. Pro urychlení tohoto procesu se používá **TLB**.

TLB (Translation Look-aside Buffer) Jedná se o rychlou **asociativní (obsahem adresovatelná paměť)** vyrovnávací paměť, která umožňuje **paralelně** (nebo částečně paralelně) **vyhledat** na základě **čísla stránky** (1. log. adresy) odpovídající **číslo rámce** (1. fyz. adresu). **TLB** obsahuje **vybrané** záznamy (zejména **číslo stránky a číslo rámce**, příznaky nemusí všechny) z tabulek stránek, **rozhodně neobsahuje** data stránek/rámců. Výběr uložených položek v **TLB** je **zásadní** pro rychlou práci s pamětí, při **TLB hit** lze ihned pomocí vyhledaného čísla rámce přistoupit (**1 přístup**) k fyzické paměti. Naopak při **TLB miss** (požadované číslo stránky není v TLB) je nutno provést **dva přístupy** do paměti, a to vyhledání čísla rámce v tabulce stránek a přístup do vyhledaného rámce (doba pro vyhledání v TLB je řádově menší než doba pro přístup k RAM). V praxi lze naštěstí díky **časové a prostorové lokalitě** zajistit **TLB hit** kolem **98%, 99%**, procesory **preemptivně** načítají očekávané mapování do TLB. K **TLB miss** může teoreticky dojít **až 4x** při provádění **jedné instrukce** (instrukce je na rozmezí 2 stránek, operandy jsou na rozmezí 2 stránek). Pozor, **neplést si TLB miss s výpadkem stránky** (tj. stránce není přidělen rámec). TLB miss lze řešit:

- **HW** automaticky hledá v tabulkách stránek (rychlejší ale dražší),

- řízení je předáno **SW**, které do TLB doplní požadovanou dvojici a hledání v TLB se opakuje.

Při **přepnutí kontextu** (změna procesu, který je prováděn) je nutné **změnit i položky v TLB** (oba procesy používají stejný LAP → vznik kolizí). Lze řešit označením některých stránek za **globální** (sdílené mezi procesy) nebo **přidáním identifikátoru procesu** do TLB (pak nemusí být záznamy odstraňovány, ale je složitější porovnání a vyžaduje více paměti). Záznamy z TLB musí být odstraněny i při změně mapování.

Hierarchické tabulky stránek

Eliminuje problém s **nadměrnou velikostí tabulek stránek**, na moderních procesorech jsou tabulky stránek **4úrovňové** (tj. strom o výšce 4, v paměti jsou uloženy jen **podstromy**, které jsou potřeba). Údaje o mapování **stránek s daty** jsou obvykle v listové úrovni, mohou být ale i v **některé z vyšších úrovní** (podle příznaku), pak má stránka a odpovídající rámec větší velikost (2 MiB nebo někdy i 1 GiB). U hierarchických tabulek **roste význam TLB**, při **TLB miss** je nutné vyhledávat ve stránkové hierarchii (až 4 přístupy do paměti navíc). Optimalizace např. použitím **různých TLB pro kód a pro data**. Pozor na výpočet fyzické adresy viz obrázky níže pro 2 úrovněovou tabulku (pro 4 je to podobné).

Hashované tabulky stránek

Tento princip je založený na **hashovací tabulce s explicitním řetězením synonym** pomocí seznamu (položky seznamu již nemusí obsahovat celé číslo stránky, pokud hashovací funkce dokáže vždy některé bity rozlišit). **Délka seznamů může být omezena**, pak je nenalezení řízeno SW (a některá překladová dvojice může být ze seznamu odebrána). Hashovací tabulka může být **pro každý proces** nebo **sdílená** (překladové položky pak musí obsahovat i číslo procesu).

Invertovaná tabulka stránek Pro **každý rámec** udává, **jaký proces** do něj má namapovanou **jakou stránku** (některé rámce mohou být aktuálně neobsazené). Index v tabulce rámců odpovídá číslu rámce (není ale problém s velikostí, protože tabulka je jen jedna pro všechny procesy). Pro urychlení vyhledávání se používá hashování. **Problémem je sdílení paměti mezi procesy** (lze řešit mapováním regionů).

Stránkování a segmentace na žádost Je nemožné (u 32 bit teoreticky ano, u 64 bit ne), aby byl celý LAP jednoho procesu, natož více procesů, namapován do paměti (RAM má obvykle kolem 8-128 GB, LAP má u 64 bit procesoru 2^{64} adres, což je mnohem víc než 128 GB). I když ale **není celý LAP procesu namapován** do fyzické paměti, **proces o tom neví**. Stránky/segmenty jsou mapovány na rámce a tím **zaváděny do RAM**, až když je to **potřeba** (proces čte/zapisuje do dané oblasti paměti). Namapování stránky na rámec je ale **časově velmi náročná** operace (může být nutné číst data z disku), proto se tyto operace snažíme eliminovat, procesor tak **preemptivně načítá stránky** do paměti (mapuje na rámce) předtím, než z nich proces chce číst/zapisovat. Při využití všech rámců je nutné část namapovaných dat odložit na disk (swap area), což většinou vede na velké zpomalení systému. Informace o načtených a odložených stránkách si vede **jádro ve svých strukturách** (také v paměti, ale zde si jádro zajistí, že tam jsou neustále), **MMU** o nich **nemá informace**. K **výpadku stránky** dochází, když do ní proces **odkazuje**, ale není načtena v tabulkách stránek nebo chybí v hashovací tabulce či invertované tabulce. Při výpadku generuje **MMU přerušení**, a jádro musí výpadek stránky obsloužit (některé stránky jsou chráněny proti výpadku, např. ty, které obsahují tabulky stránek).

Obsluha výpadku stránky

1. Kontrola, zda proces **neodkazuje mimo přidělený prostor** (o který zažádal např. použitím malloc).

2. **Alokace rámce:** buď je nějaký **prázdný** nebo musí být **vybrána oběť** (dle nějaké strategie), pokud je navíc obsah oběti modifikován (dirty bit), musí být uložen na disk - **odložen do swap**.
3. **Inicializace rámce:** ve všech případech musí být data, která v rámci zbyla po předešlém procesu **znehodnocena** aby proces, kterému je rámec nově přiřazen, toho **nemohl zneužít**. Pokud se jedná o kód, nějaká statická data nebo byla dřív stránka odložena na swap (byla editována), je rámec **přepsán** tímto, jinak je **nulován**.
4. Aktualizace tabulky stránek aby odpovídala provedené změně.
5. Proces může opakovat provedení instrukce, která způsobila výpadek. [[media/szz-38/media/image20.png]]

Může dojít k **výpadku i několika stránek** při provádění jedné instrukce (kód instrukce je na rozhraní dvou stránek, operandy jsou na rozhraní dvou stránek a ještě k tomu může dojít k výpadku v hierarchické struktuře tabulky stránek). Nejvyšší úroveň tabulky stránek musí být ale chráněna proti výpadku, jinak by nebylo možné výpadek obsloužit.

Algoritmy pro výběr oběti

Výběr odkládané stránky může být:

- **lokální** v rámci procesu, u kterého došlo k výpadku,
- **globální** tj. uvolnění libovolného rámce, nehledě na to, kdo jej používá.

Typicky je ale udržován (**page daemon** - spouští se v okamžiku nedostatku) určitý počet volných rámců. Při výpadku stránky se používá **volný rámec**. Navíc pokud byla vybrána nesprávná oběť (tj. proces požaduje stránku hned po jejím označení za oběť), je mu stránka **vrácena** a vybírá se jiná oběť. Při výběru oběti jsou **upřednostňovány nemodifikované** stránky (není je třeba odkládat na swap). Proces musí mít vždy přidělen minimální počet rámců pro provedení jedné instrukce, jinak musí být zastaven (docházelo by k neustálým výpadkům). Počet rámců přidělených procesu se určuje na základě heuristik (např. priorita, velikost programu, frekvence výpadků, ...).

FIFO (first in first out)

Odstraňuje stránku, která byla do paměti **zavedena před nejdelší dobou**.

- **výhody:** jednoduchá implementace,
- **nevýhody:** může odstranit stránku, která je namapovaná dlouho, ale také se často používá (eliminace použití heuristiky viz výše). Trpí Beladyho anomálií, tj. při zvětšení počtu rámců se může zvýšit počet výpadků.

LRU (Least Recently Used)

Odkládá **nejdéle nepoužitou stránku**. Dobře aproximuje ideální algoritmus výběru oběti (ten, který vybere stránku, která již nebude použita, nebo použita za nejdelší dobu). Vyžaduje **podporu v HW**, nutné uchovávat **časová razítka** nebo stránky mít **seřazené** podle přístupu. Obvykle se aproximují časová razítka několika bity pro každou stránku. **Jádro** periodicky stránky prochází a **nuluje bity** (shiftem doprava), při **přístupech** ke stránce se naopak nastavují **bity na 1 (MSB)**. Oběť je taková, která má v registru nejmenší hodnotu (nejvíc MSBs je na 0). Lze použít pouze jeden bit (**referenční bit**), který je periodicky nulován jádrem a nastavován při odkazu na stránku, oběť je první stránka s nulovým referenčním bitem.

- **výhody:** způsobuje malý počet výběrů nesprávné oběti.
- **nevýhody:** vyžaduje HW podporu, nefunguje pro cyklické průchody rozsáhlých polí (lze však detekovat a použít **MRU** strategii **Most Recently Used**).

Trashing

Problém, který nastává při nadměrném vytížení RAM a nedostatku rámců. **Řešení výpadků stránek zabírá delší čas, než požadovaný výpočet**, a to mnohonásobně. Řešením je úplné pozastavení některých procesů a přesunu veškeré jejich paměti na swap area.

[[media/szz-38/media/image11.png]]

Sdílení stránek

Používá se zejména pro běžné knihovny **.dll** a **.so**. Ve **FAP jsou** knihovny načteny **pouze jednou**, využívá je současně ale více procesů, více LAP je namapované na stejný FAP. Sdílení stránek také umožňuje Inter Process Communication (IPC), tj. dva procesy používají stejný úsek **FAP**. Sdílení paměťově mapovaných souborů.

Paměťově mapované soubory

Bloky souborů z disku jsou **mapovány do stránek v paměti** a mohou být pak čteny/zapisovány běžným přístupem do paměti na místo read()/write(). Umožňují sdílený přístup k souborům.

Zdroje

- SZZ okruh 38 — studijní materiály FIT BUT (szz-38.docx). Obrázky: media/szz-38/.

Navigace: [[index| • Přehled okruhů]] · □ [[okruhy/37-webova-rozhrani-autentizace|37. Webová rozhraní a autentizace]] · další: [[okruhy/39-planovani-synchronizace-procesu|39. Plánování a synchronizace procesů]] □